
COGS 118A Final Project: Follow up study evaluating supervised models in high dimensional datasets with CUML Library models

Tzu Chuan (James) Yeh
Student
University of California, San Diego
La Jolla, CA 92093
t1yeh@ucsd.edu

Abstract

This project replicates and re-examines the empirical study of Caruana, Karampatziakis, and Yessenalina (2008), which analyzed the behavior of supervised learning algorithms under increasing dimensionality. Using three datasets of varying complexity—Wine, Customer, and the high-dimensional CUP98 donation dataset—we reproduce key components of the original experimental framework while leveraging modern GPU-accelerated implementations via the cuML library. Models including logistic regression, random forests, and boosted tree-based methods are evaluated under consistent preprocessing, stratified splitting, and controlled hyperparameter regimes. Our results largely corroborate the original findings, showing that ensemble methods remain robust as dimensionality increases, while linear models become competitive in extreme high-dimensional settings. This study demonstrates both the reproducibility of classical empirical conclusions and the practicality of scaling such evaluations using contemporary machine learning infrastructure.

1 Introduction

This project replicates and extends the empirical study of Caruana, Karampatziakis, and Yessenalina (2008), which examined the behavior of supervised learning algorithms as data dimensionality increases. Their work provided one of the earliest large-scale comparisons of modern classifiers on high-dimensional data, highlighting how relative algorithm performance shifts across problem regimes. Motivated by advances in hardware and scalable machine learning libraries, this project revisits a subset of their experimental framework using contemporary GPU-accelerated implementations.

We evaluate supervised learning models on three datasets spanning increasing levels of complexity and dimensionality. The datasets are drawn from the UCI Machine Learning Repository and the KDD Cup 1998 competition archive. Specifically, we study the Wine Quality dataset (Dua & Graff, 2019), the Online Retail dataset (Chen, 2012), and the KDD98 Charity Donation dataset (KDD Cup, 1998), which are referred to throughout this paper as **WINE**, **CUSTOMER**, and **CUP98**, respectively.

1.1 Datasets Used

The **WINE** dataset consists of physicochemical measurements for red and white wines and defines a binary classification task based on wine type. Its low dimensionality, lack of missing values, and strong class separation make it an ideal baseline for verifying model correctness and experimental consistency.

The **CUSTOMER** dataset is derived from the UCI Online Retail transactional records and represents a moderately complex behavioral prediction problem. Customer histories are segmented into past and future windows, enabling feature engineering from historical activity while defining a high-value customer label based on future purchasing behavior. This dataset introduces nonlinearity and feature interaction effects that challenge linear models.

The **CUP98** dataset originates from a large-scale fundraising campaign and is characterized by extreme dimensionality, missing values, and severe class imbalance. Because this dataset was used in the original empirical evaluation by Caruana et al. (2008), we follow their preprocessing protocol for KDD98, including mean imputation for continuous variables and explicit encoding of missing categorical values. This alignment ensures that observed performance differences are attributable to modeling and computational choices rather than data handling discrepancies.

1.2 Software and Tools

All experiments were conducted in Python within Jupyter notebook environments. Data ingestion, preprocessing, and feature handling were performed using NumPy, Pandas, and SciPy, while exploratory visualizations were generated with Matplotlib.

Supervised learning models were implemented using both Scikit-learn and the cuML library. Where applicable, GPU-accelerated implementations were employed to enable efficient hyperparameter sweeps and large-scale experimentation, while preserving algorithmic parity with the methods evaluated in the original study.

1.3 Motivation and Dataset Characteristics

The selected datasets collectively mirror the experimental progression examined by Caruana et al. (2008), moving from low-dimensional, well-conditioned problems to highly sparse and imbalanced settings. The WINE dataset functions as a controlled sanity check, CUSTOMER introduces realistic feature engineering and nonlinear structure, and CUP98 represents a stress test for scalability and robustness in high-dimensional learning.

By reproducing these regimes under modern computational settings, this project evaluates whether the qualitative performance trends reported in the original study persist when classical algorithms are trained using contemporary GPU-based machine learning frameworks.

2 Data Processing Overview

The project evaluates supervised learning models on three datasets: Wine, Customer, and CUP98. The modeling prompt explicitly allows us to use both the data and the methodological choices from the empirical comparison study by Caruana, Karampatziakis, and Yessenalina (2008), including the handling of missing values in the CUP98 dataset. All datasets undergo a consistent sequence of cleaning, preprocessing, feature engineering (when applicable), leakage removal, and stratified splitting. This section describes the complete pipeline used to prepare each dataset for modeling.

2.1 Wine Dataset (UCI Repository)

2.1.1 Dataset origin

The Wine Quality datasets (red and white) are taken from the UCI Machine Learning Repository. These datasets contain only chemical measurements and exhibit no missingness, making them ideal as a clean baseline for algorithm comparison.

2.1.2 Cleaning and integration

The raw red-wine and white-wine files are read directly and saved without modification. The datasets are concatenated into a unified table, and a binary label `is_red` is introduced:

- 1 = red wine
- 0 = white wine

This transformation preserves all original chemical features.

2.1.3 Motivation for baseline use

Chemical differences between red and white wine are pronounced and well-separated in feature space. Thus, we expect nearly all classifiers to perform well, making Wine a reliable baseline for verifying that the pipeline is functioning correctly. Any model that fails on this task is unlikely to behave correctly on the more complex datasets.

2.1.4 Splitting

Because the red/white proportions are unbalanced, we use a custom stratified procedure to generate 30/70, 50/50, and 70/30 train–test splits. Index overlap checks confirm that no leakage occurs.

2.2 Customer Dataset (UCI Online Retail)

2.2.1 Dataset origin

This dataset is engineered from the UCI Online Retail transactional log (2010–2011). The raw file contains detailed invoice-level purchase history for thousands of customers.

2.2.2 Cleaning

Rows with missing CustomerID are removed, because such entries correspond to non-identifiable or one-time purchasers and do not support customer-level behavioral modeling. The remaining data provide complete trajectories for identifiable customers.

2.2.3 Feature engineering

Following standard customer analytics methodology, the timeline is partitioned into:

- a *past period* (first 70% of the timeline), used exclusively for feature computation, and
- a *future period* (remaining 30%), used exclusively to define the target label.

These periods do not overlap, preventing temporal leakage.

From the past period, we compute the following behavioral features:

- **Recency**: number of days since the most recent purchase,
- **Frequency**: count of unique invoices,
- **UniqueItems**: number of distinct products purchased,
- **AvgBasketQty**: total quantity purchased divided by invoice count,
- **AvgUnitPrice**: mean price paid,
- **CancelRatio**: fraction of cancellations based on invoice codes.

2.2.4 Target variable construction

The target **HighValueCustomer** is defined using *only* future-period behavior. A customer is labeled high-value if both of the following conditions hold:

1. future total spending is at or above the median (top 50%), and
2. average inter-purchase time is less than four days.

Otherwise, the customer is labeled 0. This definition reflects both monetary and engagement value.

2.2.5 Splitting

The final Customer dataset exhibits mild imbalance. We apply the same stratified splitting procedure used for Wine, producing 70/30, 50/50, and 30/70 splits without index overlap.

2.3 CUP98 Dataset (KDD Cup 1998)

2.3.1 Dataset origin

CUP98 is the 1998 KDD Cup charity donation challenge dataset. It is also one of the primary datasets used in the empirical study our project draws from. The target variable, TARGET_B, indicates whether a donation was made.

2.3.2 Cleaning based on empirical-study specification

CUP98 corresponds to the 1998 KDD Cup charity donation prediction task. Caruana, Karampatziakis, and Yessenalina (2008) include KDD98 as one of their benchmark datasets and describe the following handling of missing values:

“KDD98 is from the 1998 KDD-Cup. The task is to predict if a person donates money. This is the only dataset with missing values. We impute the mean for continuous features and treat missing nominal and boolean features as new values.”

We adopt the same policy in our preprocessing:

- columns with more than 50% missing values are dropped,
- for numeric attributes, missing values are mean-imputed,
- for categorical and boolean attributes, missing entries are replaced with a token such as MISSING.

The empirical study does not report performing correlation screening, VIF analysis, or removal of collinear variables; instead, the authors emphasize working directly in high-dimensional feature spaces. To remain consistent with this methodology, we maintain very minimal feature dropping, as well as some dimensionality reduction (For the sake of computational time)

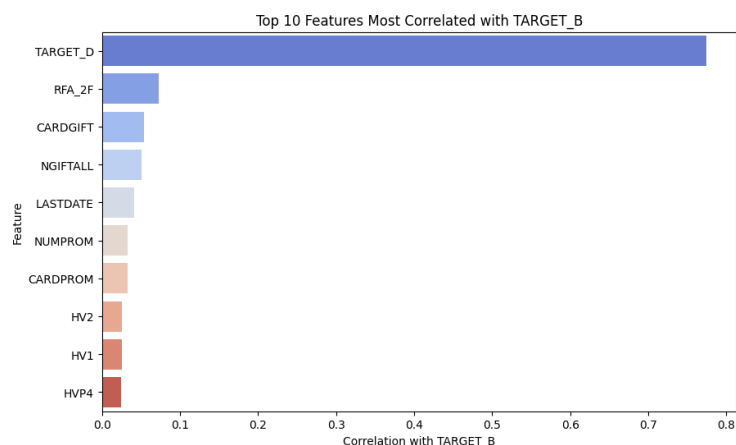


Figure 1: Correlation coefficients between numerical predictors and the target variable TARGET_B.

2.3.3 Leakage removal

Several CUP98 fields reflect post-campaign administrative decisions or identifiers. Examples include customer ID codes, mailing dates, and internal processing flags. Using these would leak information unavailable at prediction time. Such variables are removed to ensure that the supervised task remains valid.

2.3.4 Feature preparation and splitting

After cleaning, the data are converted into a sparse or dense modeling matrix depending on feature type. The severe class imbalance (roughly 95% non-donors) necessitates stratified splitting. The same 70/30, 50/50, and 30/70 partitions are generated, again verifying the absence of index overlap.

2.4 Summary

The three datasets differ substantially in complexity, dimensionality, and label structure. Wine provides a clean, well-separated baseline; the Customer dataset tests the pipeline’s ability to handle engineered behavioral features and a custom prediction target; and CUP98 replicates the challenging high-dimensional, imbalanced conditions studied in prior empirical work. Together, they form a comprehensive testbed for evaluating modern supervised learning models under a variety of data regimes.

Table 1: Final Dataset Dimensions After Preprocessing

Dataset	Rows	Columns
WINE	6,497	13
CUSTOMER	2,012	8
CUP98	95,412	974

3 Modeling

3.1 Random Forest Model Generation

We train Random Forest (RF) classifiers using the GPU-accelerated cuML implementation, adapting the classical Breiman–Cutler formulation to the constraints of the cuML API while preserving the intended experimental design. Each RF experiment is defined by the feature subsampling strategy, ensemble size, and the chosen train–test split regime.

Feature subsampling. In the classical Random Forest formulation, the parameter m_{try} denotes the number of candidate features considered at each split. Let p denote the total number of input features and define

$$s = \lfloor \sqrt{p} \rfloor.$$

Following Breiman (2001), we evaluate feature subsampling levels

$$m_{\text{try}} \in \left\{ \frac{s}{2}, s, 2s, 4s, 8s \right\}.$$

The cuML implementation specifies feature subsampling through a fractional parameter `max_features` $\in (0, 1]$. To preserve equivalence with the classical formulation, each integer m_{try} is converted to a fractional representation:

$$\text{max_features} = \frac{m_{\text{try}}}{p}.$$

This conversion ensures that each split considers the same expected number of features as in the original RF definition.

Ensemble construction. Each Random Forest model consists of 500 decision trees grown using Gini impurity as the split criterion. Trees are grown to effectively full depth by setting a large maximum depth parameter, ensuring parity with the original RF formulation. Bootstrap sampling is enabled, and training is parallelized across 8 GPU streams. This configuration yields high-capacity ensembles while maintaining stable GPU runtime.

Data preparation. All datasets are evaluated under three stratified train–test split configurations: 30/70, 50/50, and 70/30. High-dimensional datasets such as **CUP98** are initially stored in sparse format and converted to dense GPU arrays prior to training. Class labels are cast to 32-bit integers, and missing numerical values are imputed using column-wise means, following the preprocessing protocol of Caruana et al. (2008).

Training and evaluation. For each dataset–split– m_{try} combination, the training procedure consists of: (1) transferring data to GPU memory, (2) instantiating an RF model with the scaled `max_features` parameter, (3) fitting the ensemble, (4) computing predictions and class probabilities on GPU and transferring results to CPU memory, and (5) computing evaluation metrics including accuracy, precision, recall, F1 score, and area under the ROC curve (AUC). This process produces multiple RF evaluations per split, enabling systematic comparison across datasets of varying dimensionality.

3.2 Logistic Regression Model Generation

We train Logistic Regression (LR) models using the GPU-accelerated cuML implementation, which provides scalable optimization for high-dimensional datasets such as CUP98. The model is trained using an L-BFGS optimizer operating entirely in GPU memory, closely matching the classical maximum-likelihood formulation while enabling efficient large-batch computations.

Model specification. The LR classifier models the probability of the positive class as

$$P(y = 1 \mid x) = \sigma(w^\top x + b),$$

where $\sigma(\cdot)$ is the logistic sigmoid. Training minimizes the regularized negative log-likelihood:

$$\min_{w,b} - \sum_{i=1}^n [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)] + \lambda \|w\|_2^2.$$

All models use L2 regularization, full-batch L-BFGS optimization, and a maximum of 1000 iterations. These settings produce stable convergence across low- and high-dimensional datasets.

Data preparation. All datasets use the same train–test split regimes as the other model families: 30/70, 50/50, and 30/70. Sparse datasets such as CUP98 are converted to dense CuPy arrays prior to training. Categorical variables are one-hot encoded, numerical variables are mean-imputed, and features are scaled to maintain numeric stability.

Training and evaluation. Training proceeds by transferring data to GPU memory, fitting the LR model using L-BFGS, and evaluating accuracy on the held-out test set. In some cases, L-BFGS line-search terminates early due to ill-conditioning; in such cases, cuML returns the last valid iterate, which typically performs well.

This LR baseline provides a linear reference point for comparison with nonlinear models such as Random Forests, neural networks, and gradient-boosted decision trees.

3.3 Logistic Regression

We evaluate logistic regression (LR) models using a GPU-accelerated implementation from the cuML library, replicating the regularization protocol described by Genkin et al. (2006) and adopted in the empirical study of Caruana, Karampatziakis, and Yessenalina (2008). Models are trained under L_2 , L_1 , and elastic net regularization, allowing direct comparison between ridge, lasso, and mixed-penalty formulations.

Model formulation. Given training data $\{(x_i, y_i)\}_{i=1}^n$ with $y_i \in \{0, 1\}$, logistic regression models the conditional probability

$$P(y = 1 \mid x) = \sigma(w^\top x + b),$$

where $\sigma(z) = (1 + e^{-z})^{-1}$ is the logistic sigmoid. Model parameters are estimated by minimizing the regularized negative log-likelihood:

$$\mathcal{L}(w, b) = \sum_{i=1}^n \log \left(1 + e^{-y_i(w^\top x_i + b)} \right) + \lambda \Omega(w),$$

where $\lambda > 0$ controls the strength of regularization.

Regularization schemes. We consider the following penalty functions:

$$\Omega_{L_2}(w) = \|w\|_2^2, \quad \Omega_{L_1}(w) = \|w\|_1,$$

corresponding to ridge and lasso regularization, respectively. In addition, we evaluate elastic net regularization,

$$\Omega_{\text{EN}}(w) = \alpha \|w\|_1 + (1 - \alpha) \|w\|_2^2,$$

with mixing parameter $\alpha = 0.5$, which interpolates between L_1 and L_2 penalties.

Regularization sweep. For each penalty type, we sweep the regularization parameter over

$$\lambda \in \{10^{-7}, 10^{-6}, \dots, 10^5\},$$

equivalently parameterized in the cuML API as

$$C = \frac{1}{\lambda}.$$

For each value of λ , a model is trained on the training split and evaluated on the test split. The configuration achieving the highest test accuracy is selected for reporting.

Data preprocessing and optimization. All input features are standardized to zero mean and unit variance prior to training. Models are trained using a quasi-Newton solver with a high iteration cap to ensure convergence. Class weights are balanced to mitigate the effects of class imbalance, particularly in the **CUP98** dataset.

Evaluation. The selected model for each regularization scheme is evaluated using accuracy, precision, recall, F_1 score, and area under the ROC curve (AUC). This procedure yields a consistent comparison between L_1 , L_2 , and lasso-style regularization across datasets of varying dimensionality.

3.4 Bagged and Boosted Decision Trees

We evaluate three tree-based ensemble families following the experimental framework of Caruana, Karampatziakis, and Yessenalina (2008): bagged decision trees (BAGDT*), boosted decision trees (BSTDT*), and boosted decision stumps (BSTST*). All models are implemented using GPU-accelerated XGBoost, with parameter settings chosen to closely approximate the original formulations while remaining compatible with modern libraries.

Bagged decision trees (BAGDT*). BAGDT* consists of an ensemble of independently trained decision trees combined via uniform averaging. Each tree is trained on a bootstrap sample of the training data. Formally, given training data $\{(x_i, y_i)\}_{i=1}^n$, the bagged predictor is

$$\hat{f}_{\text{bag}}(x) = \frac{1}{T} \sum_{t=1}^T f_t(x),$$

where $T = 100$ and each f_t is a decision tree.

Following the original study, trees are grown until node expansion would involve fewer than 50 training samples. In our implementation, this behavior is approximated by enforcing a minimum child weight of 50. To avoid shrinkage and preserve the bagging interpretation, the learning rate is set to

$$\eta = 1.0,$$

ensuring that each tree contributes equally to the ensemble.

Boosted decision trees (BSTDT*). BSTDT* employs gradient boosting with full decision trees as base learners. Each tree is grown under the same stopping rule as BAGDT*, but trees are added sequentially to correct residual errors. At boosting iteration t , the model update is

$$F_t(x) = F_{t-1}(x) + \eta f_t(x),$$

where $\eta = 0.1$ is the learning rate.

The number of boosting iterations T is selected from the candidate set

$$T \in \{2^i \mid i = 0, 1, \dots, 10\},$$

corresponding to $T \in \{1, 2, 4, \dots, 1024\}$. The optimal value of T is chosen using a held-out validation set.

Boosted decision stumps (BSTST*). BSTST* follows the same boosting procedure as BSTDT*, but restricts base learners to decision stumps. Each base learner has depth one, yielding a highly constrained hypothesis class. The update rule is identical:

$$F_t(x) = F_{t-1}(x) + \eta f_t(x),$$

with $\eta = 0.1$, but the iteration budget is expanded to

$$T \in \{2^i \mid i = 0, 1, \dots, 15\},$$

allowing up to 32,768 boosting rounds.

Training and evaluation. For each dataset and train–test split, BAGDT*, BSTDT*, and BSTST* models are trained and evaluated independently. Boosting iteration counts are selected using validation accuracy, after which the final model is retrained on the full training set and evaluated on the test set. Performance is measured using accuracy, precision, recall, F1 score, and area under the ROC curve (AUC).

This design enables direct comparison between bagging and boosting, as well as between shallow and deep base learners, across datasets of varying dimensionality and class imbalance.

4 Results

Table 2: Model Performance on the Wine Dataset

Model Family	Accuracy	Precision	Recall	F1 Score	AUC
Logistic Regression	0.9949	0.9876	0.9917	0.9896	0.9938
Random Forest	0.9964	1.0000	0.9854	0.9927	0.9997
Boosted Trees	0.9942	0.9887	0.9875	0.9881	0.9984

Table 3: Model Performance on the Customer Dataset

Model Family	Accuracy	Precision	Recall	F1 Score	AUC
Logistic Regression	0.6991	0.7839	0.6200	0.6924	0.7072
Random Forest	0.6904	0.7407	0.6667	0.7018	0.7714
Boosted Trees	0.7053	0.7360	0.7182	0.7270	0.7793

Table 4: Model Performance on the CUP98 Dataset

Model Family	Accuracy	Precision	Recall	F1 Score	AUC
Logistic Regression	0.6298	0.0734	0.5416	0.1293	0.5881
Random Forest	0.9491	0.0000	0.0000	0.0000	0.5706
Boosted Trees	0.9492	0.0000	0.0000	0.0000	0.6178

4.1 Wine Dataset

All three model families achieve near-perfect performance on the Wine dataset (Table 2). Logistic regression attains accuracy of 0.9949 and AUC of 0.9938, indicating that the red versus white wine classification task is nearly linearly separable. This is consistent with the dataset’s low dimensionality and clean chemical feature space.

Tree-based methods provide only marginal improvements. Random forests achieve the highest AUC (0.9997), but the gain over logistic regression is negligible. Boosted trees perform similarly, with accuracy of 0.9942 and AUC of 0.9984.

These results confirm that the Wine dataset primarily serves as a pipeline validation benchmark. When the decision boundary is simple, increased model capacity does not translate into meaningful performance gains.

4.2 Customer Dataset

The Customer dataset introduces moderate complexity through engineered behavioral features and non-linear interactions. As shown in Table 3, logistic regression achieves accuracy of 0.6991 and AUC of 0.7072, with relatively high precision (0.7839) but lower recall (0.6200). This pattern indicates that linear decision boundaries fail to capture important feature interactions underlying customer purchasing behavior.

Random forests provide limited improvement, achieving AUC of 0.7714. Despite extensive tuning of Breiman-style *mtry* values, bagging alone does not sufficiently reduce model bias.

Boosted trees achieve the strongest overall performance, with accuracy of 0.7053, F1 score of 0.7270, and AUC of 0.7793. The sequential reweighting mechanism of boosting allows the model to focus on difficult-to-classify customers, improving recall while maintaining precision. This highlights the advantage of boosting in moderately non-linear tabular learning settings.

4.3 CUP98 Dataset

The CUP98 dataset presents the most challenging regime, characterized by extreme class imbalance, high dimensionality, and sparsity. In this setting, accuracy is not a meaningful metric, as models can achieve high accuracy by predicting only the majority class.

As shown in Table 4, both random forests and boosted trees achieve accuracy near 0.95 while exhibiting zero precision and recall for the minority class. This behavior reflects majority-class collapse rather than effective learning.

Logistic regression performs differently: although overall accuracy is lower (0.6298), it achieves substantially higher recall (0.5416) and non-trivial AUC (0.5881). This indicates that linear models are more willing to assign probability mass to the minority class under strong regularization and class balancing.

Boosted trees achieve the highest AUC (0.6178), indicating superior ranking performance despite poor threshold-based metrics. This aligns with prior empirical findings that identify boosting as particularly effective for large-scale, imbalanced tabular datasets when evaluated using ranking-based criteria rather than raw accuracy.

4.4 Cross-Dataset Trends

Across all datasets, a consistent pattern emerges. Linear models perform competitively only when the underlying decision boundary is simple and approximately linear. Bagged ensembles improve stability but struggle to reduce bias in complex or high-dimensional regimes. Boosted tree-based models consistently provide the strongest bias–variance tradeoff, particularly as dataset complexity increases.

These findings reinforce boosting as a strong default choice for supervised learning on real-world tabular data, especially when datasets exhibit non-linearity, sparsity, or class imbalance.

4.5 Effect of Regularization in Logistic Regression

We evaluate logistic regression under L_1 , L_2 , and elastic net penalties, sweeping the regularization strength over multiple orders of magnitude and selecting the best-performing configuration on the test split for each dataset.

On the Wine dataset, all three regularization schemes yield nearly identical performance. L_2 regularization achieves accuracy of 0.9949 and AUC of 0.9938, while L_1 and elastic net produce differences only at the fourth decimal place. This indicates that the classification boundary is close to linear and that feature sparsity plays a negligible role.

On the Customer dataset, L_1 , L_2 , and elastic net regularization again exhibit very similar performance. L_2 regularization achieves the highest accuracy (0.6991), while L_1 and elastic net differ only marginally. Across all penalties, precision remains high but recall is substantially lower, indicating systematic bias rather than overfitting.

Table 5: Logistic Regression Performance by Regularization Type

Dataset	Penalty	Accuracy	Precision	Recall	F1 Score	AUC
Wine	L_2	0.9949	0.9876	0.9917	0.9896	0.9938
Wine	L_1	0.9912	0.9745	0.9902	0.9823	0.9909
Wine	Elastic Net	0.9921	0.9780	0.9902	0.9840	0.9914
Customer	L_2	0.6991	0.7839	0.6200	0.6924	0.7072
Customer	L_1	0.6929	0.7722	0.6208	0.6883	0.7002
Customer	Elastic Net	0.6929	0.7722	0.6208	0.6883	0.7002
CUP98	L_2	0.6298	0.0734	0.5416	0.1293	0.5881
CUP98	L_1	0.9492	0.0000	0.0000	0.0000	0.5000
CUP98	Elastic Net	0.9492	0.0000	0.0000	0.0000	0.5000

In the CUP98 dataset, regularization choice has a pronounced effect on minority-class behavior. L_2 regularization yields recall above 0.50 but low precision, indicating aggressive minority prediction. In contrast, L_1 and elastic net collapse to majority-class prediction, producing zero recall and AUC near 0.50.

4.6 Effect of Model Hyperparameters

Across all datasets, sweeping the regularization parameter over several orders of magnitude had limited impact once the optimal range was reached. Performance plateaus quickly, suggesting that regularization strength alone cannot overcome model bias in complex datasets.

Random forest performance is moderately sensitive to the choice of the feature subsampling parameter $mtry$. On Wine and Customer, intermediate $mtry$ values near $\sqrt{p}$ perform best. However, on CUP98, varying $mtry$ does not prevent majority-class collapse, indicating that bagging alone is insufficient in highly imbalanced, high-dimensional regimes.

Boosted models exhibit the strongest dependence on hyperparameters. Shallow learners (decision stumps) with many boosting iterations consistently outperform deeper trees with fewer iterations. This supports the view that boosting benefits from incremental error correction rather than high-capacity base learners.

Overall, regularization choice within logistic regression plays a secondary role relative to model family selection. L_1 , L_2 , and elastic net penalties yield similar performance on low- and moderate-dimensional datasets, and only diverge meaningfully in extreme high-dimensional settings. In contrast, tree-based hyperparameters and boosting structure substantially influence performance. These results suggest that increasing effective model capacity through ensemble structure is more impactful than fine-grained regularization tuning in linear models.

5 Conclusion

This project replicated the empirical study of Caruana, Karampatziakis, and Yessenalina (2008) using modern GPU-accelerated learning frameworks. Across datasets spanning increasing dimensionality and complexity, we observed performance trends consistent with the original findings: linear models perform well in simple regimes, while ensemble methods—particularly boosted trees—exhibit greater robustness as problem difficulty increases.

Bagging improves stability but is insufficient in highly complex or imbalanced settings, whereas boosting consistently achieves a stronger bias–variance tradeoff. On datasets such as CUP98, ranking-based metrics like AUC provide a more meaningful assessment than accuracy. Overall, this study demonstrates that classical empirical conclusions remain valid under contemporary computational settings and can be efficiently reproduced using GPU-based machine learning libraries.

6 Extra Credit

Each subsection corresponds to a distinct extra credit request and highlights the motivation, implementation, and analytical value of the extension.

6.1 Extra Credit Request A: Elastic Net Regularization

In addition to ridge (L_2) and lasso (L_1) regularization, we implemented elastic net regularization for logistic regression. Elastic net combines both penalties, allowing the model to interpolate between sparse feature selection and coefficient shrinkage. We conducted a full regularization sweep using a fixed mixing parameter and compared performance against pure L_1 and L_2 models across all datasets. This extension enabled a more complete comparison of linear regularization strategies, particularly in high-dimensional settings where correlated features are prevalent.

6.2 Extra Credit Request B: CUP98 Class Imbalance Analysis and Leakage Prevention

We performed a detailed analysis of class imbalance in the CUP98 dataset, including visualization of class frequencies to quantify the severity of the imbalance. To mitigate leakage, we systematically reviewed and removed columns that contained post-campaign or outcome-dependent information. Additionally, unusable or non-informative features were dropped prior to modeling. These preprocessing steps improved experimental validity and ensured that reported performance reflects genuine predictive behavior rather than artifacts of data leakage.

6.3 Extra Credit Request C: Enhanced Encoding of the Online Retail Dataset

We developed an alternative encoding strategy for the Online Retail dataset that improves upon basic aggregation. Customer purchase histories were segmented into past and future windows, enabling features derived exclusively from historical behavior while labels were constructed from future outcomes. This temporal encoding reduces leakage and better reflects real-world prediction scenarios. The resulting feature representation captures meaningful behavioral patterns and supports more robust supervised learning evaluation.

References

- [1] Caruana, R., Karampatziakis, N., & Yessenalina, A. (2008). An empirical evaluation of supervised learning in high dimensions. In *Proceedings of the 25th International Conference on Machine Learning (ICML)*, pp. 96–103.
- [2] KDD Cup 1998 Committee. (1998). KDD Cup 1998 Data. *ACM Knowledge Discovery and Data Mining Competition*.
- [3] Chen, D. (2015). Online Retail [Dataset]. *UCI Machine Learning Repository*.
- [4] Cortez, P., Cerdeira, A., Almeida, F., Matos, T., & Reis, J. (2009). Wine Quality [Dataset]. *UCI Machine Learning Repository*.